



INDIA.RUNS

Build what next India runs on

Team Name : OctaOps

Team Leader Name : Tanmay Kiran Vishe

Problem Statement : Build an offline candidate ranking system that analyzes 1,00,000 profiles against a description and returns the Top 100 most relevant candidates with explainable reasoning.

Solution Overview

- What is your proposed solution?

An offline candidate ranking system that streams 100,000 profiles, applies a 6-signal evidence-based scoring pipeline, and returns the Top 100 most relevant candidates with factual explanations.

- What differentiates your approach from traditional candidate matching systems?

- Career history evidence analysis beyond skill keywords
- Detects production retrieval/ranking ownership
- Penalizes keyword-stuffing and trap profiles
- Dynamic evidence-based reranking
- Template-based explainability without LLM hallucination risk
- Fully offline, CPU-only, reproducible execution

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?
 - Retrieval & ranking systems
 - Vector search & embeddings
 - Production vector DB experience
 - Recommendation/search systems
 - Evaluation frameworks (NDCG, MRR, MAP, A/B testing)
 - Startup product-engineering mindset
 - 5–9 years experience range
- Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?
 - Career evidence of shipped retrieval/ranking systems
 - Technical skill overlap with JD
 - Production ownership evidence
 - Production deployment evidence
 - Evaluation maturity (NDCG/MRR/A-B testing)
 - Product-company AI experience
 - Availability & recruiter responsiveness

Ranking Methodology

- How does your system retrieve, score, and rank candidates?
 1. Parse JD to identify required skills, domain terms, and experience expectations
 2. Stream 100K candidate profiles from JSONL without loading full dataset into memory
 3. Normalize profiles and compute 6-signal weighted base score for every candidate
 4. Apply career evidence calibration using work-history text and production signals
 5. Maintain dynamic Top-1000 shortlist using min-heap for efficient candidate retrieval
 6. Stage-2 rerank shortlist with evidence depth, behavioral signals, and quality penalties
 7. Select final Top-100 candidates and generate factual reasoning for each profile
- What models, algorithms, or heuristics are used?
 - Deterministic rule-based ranking (no ML/LLM APIs)
 - Weighted scoring: skills + terms + groups + experience + execution + career evidence
 - Career-text analysis for production ownership evidence
 - Trap, keyword-stuffing & weak-evidence penalties
- How are multiple candidate signals combined into a final ranking?
 - Normalize candidate signals into weighted scores
 - Apply ± 0.10 career evidence calibration
 - Add depth bonus + behavioral tie-breakers
 - Penalize surface matches and trap profiles
 - Sort by final score + deterministic ID tie-break

Explainability & Data Validation

- How are ranking decisions explained?

Ranking decisions are explained through deterministic 1-2 sentence reasoning generated from each candidate's actual profile data.

Each explanation cites concrete evidence such as years of experience, title, company, retrieval/ranking/vector work, production signals, and availability.

No hosted LLM is used during ranking; explanations are generated from structured candidate fields to keep reasoning factual and reproducible.

- How do you prevent hallucinations or unsupported justifications?

We prevent hallucinations by generating explanations only from structured candidate data and detected career evidence.

The system does not use a hosted LLM during ranking, so The reasoning engine avoids free-form generation and only references existing profile fields such as experience, company, skills, and detected career evidence.

Every justification is tied to actual profile fields such as experience, title, employer, skills, production signals, and behavioral data.

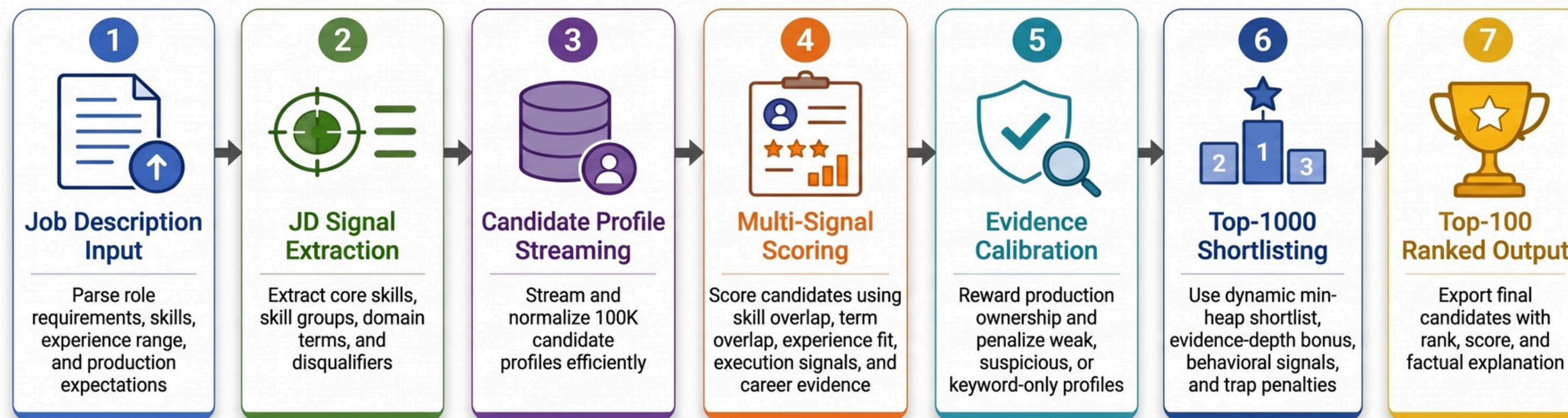
- How does your solution handle inconsistent, low-quality, or suspicious profiles?

The system normalizes every profile and penalizes suspicious patterns such as keyword stuffing, framework-only AI claims, research-only mismatch, weak production evidence and honeypot / impossible profile patterns.

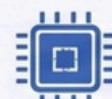
LoLow-quality profiles are not removed; they receive full scoring but penalties reduce ranking when evidence is weak.

End-to-End Workflow

- What is the complete workflow from JD input to ranked candidate output?



100K Candidates
Processed



CPU-Only
Execution



No External APIs
100% Offline

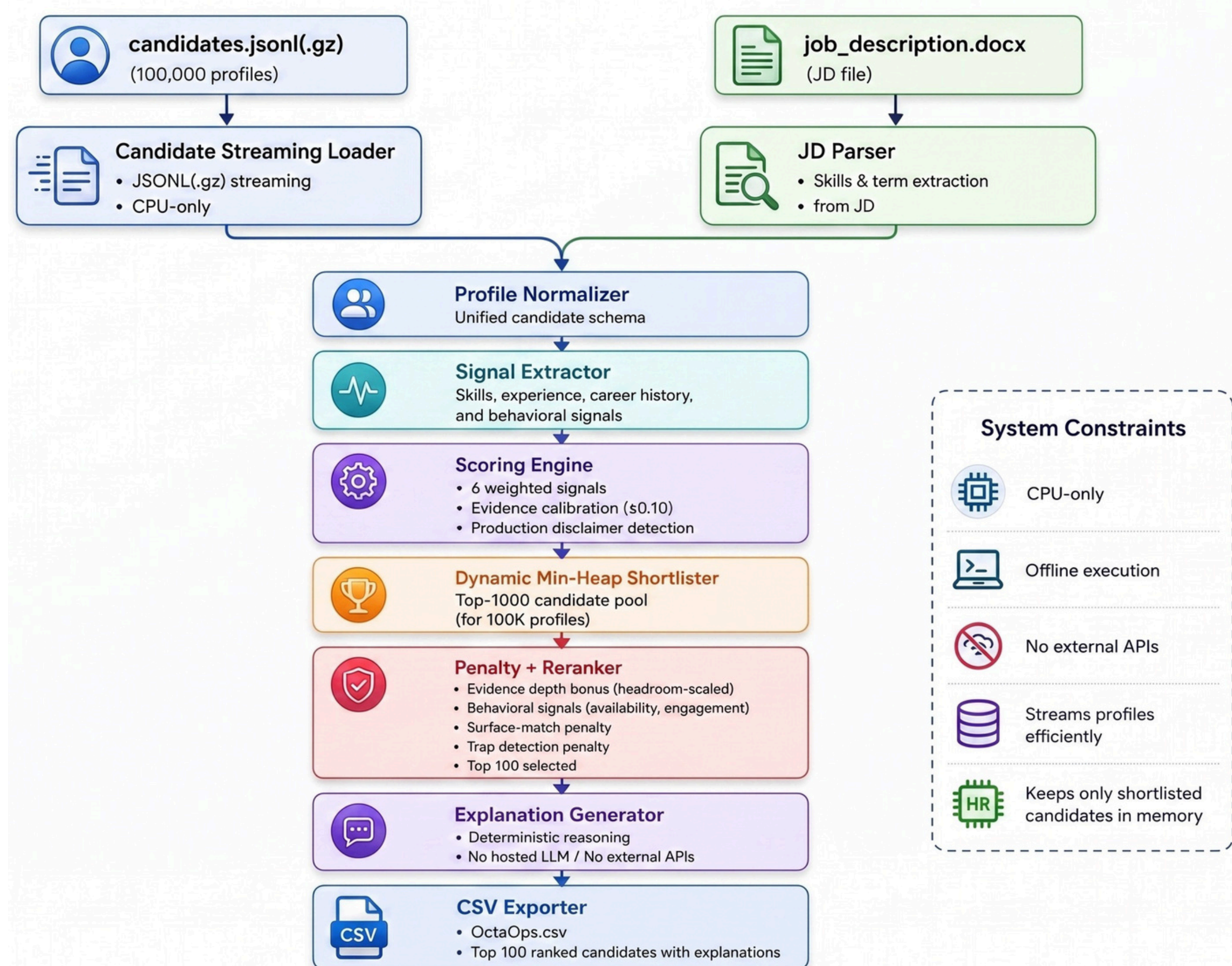


Explainable
& Auditable



Output: CSV
(Top 100)

System Architecture



Results & Performance

- What results or insights demonstrate ranking quality?
 - Career-backed top ranks: Top candidates show real work-history evidence in ranking, recommendation, embeddings, vector search, production pipelines, monitoring, and A/B testing.
 - Stronger than keyword matching: The system penalizes keyword stuffing, framework-only AI claims, research-only mismatch, and weak production ownership, improving ranking trust.
 - Validated final output: The Top-100 CSV has unique candidates, monotonic scores, factual explanations, and passes submission validation with reasoning generated only from structured profile data.
- How does your solution meet the challenge's runtime and compute constraints?
 - Runtime within limit: The full 100K-candidate ranking pipeline completes in approximately 104-105 seconds, well below the 300-second challenge limit.
 - Compute-efficient design: It runs CPU-only, makes no external API calls, streams candidates instead of loading the full dataset, and keeps only the dynamic Top-1000 candidate pool in memory.

Technologies Used

- What technologies, frameworks, and tools were used and why were they selected for this solution?
 - Python: main ranking pipeline and data processing
 - Pydantic: structured candidate validation
 - Pandas: dataset handling and normalization
 - heapq / min-heap: efficient Top-300 shortlisting
 - gzip/json/csv: offline file processing
 - Rule-based scoring engine: deterministic, auditable, and reproducible
 - Template/rule-based explanation generator: avoids hosted LLM dependency

Why selected:

- Fast enough for 100K profiles
- Works offline
- Easy to audit
- No API cost or network risk
- Deterministic results

Submission Assets

- GitHub Repository: <https://github.com/tanmay75336/talent-intelligence-ai>
- Sandbox / Demo: Google Colab reproduction environment
<https://colab.research.google.com/drive/12h8SmdQUZcTO4LkyQdWoHNU0DGOsgciX?usp=sharing>
- Reproduce Command: `python -m backend.competition.rank --candidates data/candidates.jsonl --job data/job_description.docx --output OctaOps.csv`
- Submitted Output: OctaOps.csv
- Validation: Official validator passed: "Submission is valid."



INDIA.RUNS

Build what next India runs on

THANK YOU